BrainAI

ProSuite Industrial

CORE ARCHITECTURE

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProED Engine Handbuch**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Core Engine
2.0.0	26.02.2026	S. Köhne	Stable	Hybrid Ring-0 Edition (ProKM Kernel Support & Armored Envelope)

1 Produktübersicht

Die ProED Core Engine v1.1.0 ist ein extrem performantes, hybrides Kryptografie-Modul für Hochsicherheitsumgebungen. Sie bietet eine verzweigungsfreie (branchless) Verschlüsselung und ist darauf ausgelegt, Daten sowohl im User-Space (Ring-3) als auch hardwarebeschleunigt direkt im Betriebssystem-Kernel (Ring-0 via ProKM) zu verarbeiten.

Kernmerkmale

- **Hybride Architektur:** Automatische Erkennung und Nutzung des `ProKM` Kernel-Treibers für maximale I/O-Performance. Fällt das System auf eine Umgebung ohne Treiber zurück, übernimmt nahtlos die native Shared Library (DLL/SO).
- **Armored Envelope (AEAD):** Die Software-Implementierung kapselt die Nutzdaten automatisch in einen kryptografischen Tresor ("Envelope"). Dieser enthält den Initialisierungsvektor (IV) und einen Constant-Time HMAC, um Manipulationen vor der Entschlüsselung abzuwehren.
- **Zero-Allocation im Kernel:** Im Ring-0-Modus arbeitet die Engine vollständig speichereffizient ohne dynamische Speicherallokationen (Zero-Copy-Ansätze möglich).
- **Side-Channel Immunity:** Alle Kernalgorithmen sind speicher- und zeitkonstant (Constant-Time) implementiert, um Cache- und Timing-Angriffe physikalisch auszuschließen.

2. Technische Spezifikationen

2.1 Globale Konstanten

Diese Werte sind für alle Implementierungen verbindlich und gewährleisten Speichersicherheit bei der Allokation:

- **HEADER_SIZE:** *48* Bytes (Fixe Größe des "Armored Envelope" Headers).
- **KEY_SIZE:** Exakt *32* Bytes (*256*-Bit Entropie).
- **CTX_SIZE:** *120* Bytes (Größe des internen State-Contexts im RAM).

2.2 Paket-Layout (Software Armored Envelope)

Wenn die Daten im User-Space verarbeitet werden (`EncryptEnvelopeSoftware`), generiert ProED einen durchgehenden Binärblock mit folgender Struktur:

- **Byte 00 – 15:** Initialisierungsvektor (IV) im Klartext.
- **Byte 16 – 47:** Constant-Time Auth-Tag (HMAC) zur Integritätsprüfung.
- **Byte 48 – End:** Verschlüsselte Nutzlast (Payload).

Hinweis: Im Kernel-Mode (`ProcessKernel`) werden Metadaten und IV via IOCTL-Struct übergeben und die reine Payload zurückgegeben.

3. Globales Error-Management

Die Engine nutzt ein direktes Rückgabemodell in der C-Schnittstelle, das von den Wrappern ausgewertet wird.

Code	***Bedeutung***	***Beschreibung***	***Kritikalität***
1	SUCCESS	Operation erfolgreich abgeschlossen. Payload ist authentisch.	*Keine*
0	FAILURE	Operation abgebrochen. Entweder Parameterfehler oder (bei Decrypt) ist der HMAC-Check fehlgeschlagen. Das Paket wurde manipuliert!	*Hoch (Angriff!)*

4. C# (.NET) Integration & Hybrid Handling

Der *C#-Wrapper* abstrahiert die gesamte Systemkomplexität. Er prüft beim Start via `CreateFile`, ob der Windows-Kernel-Treiber (`\\.\ProKM`) erreichbar ist, und routet die Daten intelligent.

4.1 Implementierungsbeispiel

```
using ProED;

public void HybridEncryptionExample()
{
    byte[] sessionKey = GetSecureKey32Bytes();
    byte[] iv = GetSecureRandom16Bytes();
    byte[] secretData = System.Text.Encoding.UTF8.GetBytes("Critical

    // Using-Block stellt sicher, dass Kernel-Handles (falls offen) f
    using (var proed = new ProEDClient(sessionKey))
    {
        byte[] encrypted;

        if (proed.IsKernelModeActive)
        {
            // Maximale Performance: Verarbeitung direkt in Ring-0
            LogInfo("ProKM Kernel Driver gefunden! Route Daten durch
            encrypted = proed.ProcessKernel(secretData, iv, isDecrypt
        }
        else
        {
            // Maximale Kompatibilität: Verarbeitung im User-Space mi
            LogInfo("Kein Treiber. Nutze C-DLL Fallback (Armored Enve
            encrypted = proed.EncryptEnvelopeSoftware(secretData);
        }

        if (encrypted == null)
        {
            throw new SecurityException("Verschlüsselung auf Engine-E
        }
```

```
    }
}
```

5. Python (High-Performance Scripting)

Der *Python-Wrapper* kombiniert `ctypes` mit dem `struct`-Modul, um auf Windows-Systemen native `DeviceIoControl` Aufrufe an den Treiber zu senden und auf Linux reibungslos die Shared Library (`.so`) zu nutzen.

5.1 Implementierungsbeispiel

```
from proed import ProED

def process_sensor_data(sensor_payload: bytes, key: bytes, iv: bytes)
    # Context-Manager (__enter__ / __exit__) sichert Handle-Releases
    with ProED(key) as engine:

        try:
            if engine.is_kernel_mode_active:
                print("Hardware-Beschleunigung aktiv.")
                encrypted = engine.process_kernel(sensor_payload, iv,
            else:
                print("Software-Modus aktiv.")
                encrypted = engine.encrypt_envelope_software(sensor_p

            if encrypted is None:
                raise ValueError("Kryptografischer Fehler.")

            transmit_to_cloud(encrypted)

        except OSError as e:
            print(f"I/O Fehler bei der Kommunikation mit dem Kernel:
```

6. Java (JNA / Enterprise Integration)

Die *Java*-Implementierung nutzt JNA (Java Native Access), um die Systemgrenzen der JVM zu durchbrechen. Sie lädt sowohl die ProED-Library als auch die `kernel32.dll`, um Kernel-Speicherzugriffe aus dem Java-Backend heraus zu steuern.

6.1 Implementierungsbeispiel

```
import com.brainai.proed.ProED;

public class SecurityService {
    public byte[] decryptIncoming(byte[] networkPacket, byte[] sessio
        // Try-With-Resources für IDisposable-Pattern
        try (ProED engine = new ProED(sessionKey)) {

            if (engine.isKernelModeActive()) {
                // Setzt voraus, dass der IV separat über das Protoko
                byte[] iv = extractIvFromProtocol(networkPacket);
                byte[] rawCipher = extractCipherFromProtocol(networkP
                return engine.processKernel(rawCipher, iv, true);
            } else {
                // Entschlüsselt den kompletten 48-Byte Envelope inkl
                byte[] decrypted = engine.decryptEnvelopeSoftware(net

                if (decrypted == null) {
                    System.err.println("CRITICAL: MAC-Fehler. Paket w
                    triggerSecurityAlert();
                }
                return decrypted;
            }
        }
    }
}
```

7. JavaScript (Node.js / Web-Services)

Der *Node.js* Wrapper nutzt `ffi-napi` und native Buffer. Er ist darauf ausgelegt, massiv asynchrone I/O-Streams extrem speichereffizient (Zero-Copy) an den Treiber durchzureichen.

7.1 Implementierungsbeispiel

```javascript
const ProED = require('./proed');

function secureDataStream(rawData, keyBuffer, ivBuffer) {
    const engine = new ProED(keyBuffer);
    let cipherBuffer = null;

    if (engine.isKernelModeActive) {
        console.log("Nutze Ring-0 für High-Throughput.");
        cipherBuffer = engine.processKernel(rawData, ivBuffer, false)
    } else {
        console.log("Nutze Ring-3 Fallback.");
        cipherBuffer = engine.encryptEnvelopeSoftware(rawData);
    }

    // WICHTIG: Speicher-Handle am Kernel explizit freigeben
    engine.close();

    if (!cipherBuffer) {
        throw new Error("Encryption failed.");
    }
    return cipherBuffer;
}
```

8. VB.NET (Industrial Systems)

In der industriellen Automatisierung (OT / SPS-Steuerungen) sichert der VB.NET Wrapper Anlagen gegen Manipulationen. Die `IDisposable`-Schnittstelle garantiert die Stabilität langlaufender Systeme (kein Handle-Leaking).

8.1 Implementierungsbeispiel

```
Public Sub HandleMachineCommand(packet As Byte(), key As Byte())
    ' Using Block garantiert den Aufruf von CloseHandle() auch bei Ex
    Using engine As New ProEDClient(key)

        Dim plainCommand As Byte()

        If engine.IsKernelModeActive Then
            Dim iv As Byte() = ExtractIv(packet)
            Dim cipher As Byte() = ExtractCipher(packet)
            plainCommand = engine.ProcessKernel(cipher, iv, True)
        Else
            plainCommand = engine.DecryptEnvelopeSoftware(packet)
        End If

        If plainCommand Is Nothing Then
            RaiseEvent SecurityAlert("Integritätsverletzung! Maschine
            MachineControl.EmergencyStop()
            Exit Sub
        End If

        MachineControl.ExecuteCommand(plainCommand)
    End Using
End Sub
```

9. Best Practices & Sicherheitsrichtlinien

- **Ressourcen-Management (Kernel Handles):** Der hybride Wrapper greift auf Systemebene (Ring-0) auf den `ProKM` Treiber zu. Alle Instanzen **müssen** nach Gebrauch geschlossen werden (`Dispose`, `close()`, `with`-Statement), um Memory- und Handle-Leaks im Betriebssystem zu vermeiden.
- **Nonce/IV Management:** Ein Initialisierungsvektor (IV) darf unter demselben Schlüssel *niemals* wiederverwendet werden. Beziehen Sie IVs ausschließlich aus echten Hardware-Entropiequellen (wie der `ProKey` Engine).
- **Buffer-Wiping:** Die C-Engine löscht ihre internen Zustände (`ProED_Context`) automatisch sicher aus dem RAM. In den High-Level Sprachen (C#, Java) obliegt es jedoch dem Garbage Collector, Speicher freizugeben. Hochsensible Klartext-Buffer sollten nach der Nutzung manuell mit Nullen überschrieben werden.
- **Treiber-Deployment:** Der `ProKM` Treiber muss mit Administratorrechten installiert und signiert sein. Fehlen diese Rechte zur Laufzeit, fällt der Wrapper lautlos und stabil auf die "Armored Envelope" Software-Verschlüsselung zurück.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**